ChatGPT

# Sky98: Matrix-Based Proof-of-Work for AI-Centric Blockchain Consensus

**Abstract:** *We introduce Sky98, a novel proof-of-work (PoW) protocol designed around deterministic, irreducible integer matrix computations. Unlike traditional hash-based PoW, Sky98 requires miners to perform meaningful linear algebra operations – large matrix multiplications with non-linear transformations – that align with the capabilities of modern AI accelerators (GPUs, TPUs, etc.). We formally define the Sky98 algorithm, which iteratively multiplies integer matrices and applies a non-linear $\sigma$ function, masking, and modular reduction at each step. The design enforces deterministic arithmetic (no floating-point variation) to guarantee consensus safety across different hardware. A carefully chosen non-linear $\sigma$ function breaks algebraic compressibility, ensuring the computation remains* computationally irreducible *(i.e. cannot be shortcut or algebraically simplified). Each mining epoch incorporates an identity-bound random seed to prevent replay or precomputation attacks. We describe how block verification is achieved efficiently by sampling sub-matrices (tiles) or using random linear combination techniques, bounding the verifier's cost well below the original work. We analyze why the Sky98 workload is* irreducible *– miners must perform essentially the full matrix operations to find a valid proof – and why attempts to shortcut the work are infeasible. Additionally, we examine the work-depth and memory balance of Sky98's computation, showing how it leverages massive parallelism* and *memory bandwidth in a manner that favors general-purpose AI hardware advances over brute-force ASICs. The broader impact is a PoW scheme that incentivizes innovation in general computing (fast matrix math, high-bandwidth memory) rather than energy-wasting custom hashes. We contrast Sky98's approach with the stagnation of SHA-256 ASIC mining, arguing that matrix-based PoW can revitalize hardware research while securing blockchain consensus.*

## Introduction

Blockchain proof-of-work has traditionally relied on *cryptographic hashing* puzzles (such as SHA-256 in Bitcoin) that, while effective for security, perform inherently wasteful computations [1] [2]. These hash computations have no value outside securing the blockchain and have led to specialized ASIC miners that excel at nothing but brute-force hashing. Over time, this dynamic has introduced *hardware stagnation*: enormous engineering effort and energy expenditure are devoted to SHA-256 ASICs that cannot be repurposed for any other useful computation [2]. In contrast, the broader computing world – particularly artificial intelligence – demands hardware capable of high-throughput *matrix and tensor operations*. Modern AI workloads *"typically involve multiplying huge matrices, applying the same operation to billions of parameters, and doing this millions of times"* [3]. As a result, GPUs and TPUs have evolved into extremely efficient parallel machines for linear algebra, featuring massive numbers of multiply-accumulate units and high memory bandwidth [4] [5].

This paper proposes **Sky98**, a proof-of-work algorithm that bridges these two worlds by turning blockchain mining into a workload that mirrors *useful AI computation*. Rather than hashing, Sky98 miners perform large integer **matrix multiplications** combined with non-linear transformations, closely matching the "native computation" of modern AI accelerators. The goal is to harness the *bottleneck operation of AI* – matrix multiply – for consensus, thus *"nearly eliminating the energy-waste of Bitcoin mining"* by allowing the same hardware investment to serve *dual purposes* [6]. In essence, a miner's compute power can secure the blockchain *and* advance AI computations, a "2-for-1" use of resources [6]. This addresses the *longstanding open problem* of designing a PoW based on a *real-world*

*computational task (as opposed to artificial random hashing)* [7], without compromising the permissionless and competitive nature of Nakamoto consensus.

**Sky98's approach in context:** Prior works have explored **Proof-of-Useful-Work** (PoUW) and related ideas. For example, Komargodski et al. (2023) present a PoUW scheme for arbitrary matrix multiplication, demonstrating how miners can produce a PoW certificate with negligible overhead beyond the matrix multiply itself [8]. Their protocol "cuPOW" injects carefully crafted noise into matrix products so that *verifying the computation's transcript* proves the work was done [9] [10]. Our Sky98 design shares the high-level vision of making PoW *useful* and aligned with AI, but it introduces a different algorithmic structure (deterministic iterative matrix transformations with non-linear layers) to enforce *computational irreducibility* and ease of verification. Sky98 emphasizes **deterministic integer arithmetic** for cross-platform consensus safety, and a novel $\sigma$ (sigma) function to break any algebraic structure that could allow shortcuts. We also integrate an **epoch-based seed** mechanism tied to miner identity or block context to prevent replaying or reusing work across rounds.

In the following sections, we provide a detailed, formal definition of the Sky98 PoW algorithm, breaking down each step in the matrix-based computation pipeline. We then discuss how deterministic arithmetic constraints and the design of the non-linear $\sigma$ function ensure that all miners see exactly the same problem and that the problem cannot be algebraically simplified. We describe the *non-compressibility* of the workload: intuitively, the only way to find a valid proof is to actually perform the prescribed matrix operations step by step, which is the essence of **computational irreducibility** [11]. Next, we cover the consensus-level aspects such as how the challenge seeds are formed each epoch and bound to identities to avoid replay attacks. We then detail the verification process – how a proof (which may include a resulting matrix or a digest of it) can be verified efficiently by *sampling* a small portion of the computation or using probabilistic check techniques, so that full nodes need not redo the entire matrix multiplication.

Finally, we turn to a hardware and security analysis. We examine the **work–depth** characteristics of Sky98: the algorithm has abundant parallel work (multiplying many matrix elements concurrently) but also enforced sequential depth (multiple iterative rounds), which together map well to GPU/TPU architectures while limiting single-purpose ASIC optimizations. Memory usage and data movement are considered alongside pure compute: the algorithm is designed such that both computation and memory bandwidth are critical (similar to AI workloads), narrowing the efficiency gap that a custom ASIC could achieve. We discuss how any specialized miner for Sky98 would essentially resemble an AI matrix accelerator, meaning that improvements in mining hardware translate to improvements in general-purpose AI hardware rather than a dead-end. In contrast to Bitcoin's SHA-256, where ASICs enjoy orders-of-magnitude efficiency gains over GPUs [12] [13], Sky98 aims for **ASIC-resistance** in the sense of minimizing the advantage of custom hardware [14]. The ultimate vision is a PoW consensus that *drives* progress in computing hardware (faster matrix math, better memory systems) and yields useful by-products, instead of burning electricity on opaque hashes. We close by reflecting on the broader implications for blockchain security and the co-evolution of blockchain and AI infrastructure.

# The Sky98 Proof-of-Work Algorithm

## Formal Algorithm Definition

At its core, Sky98 defines a *deterministic* function $F$ that transforms an input *challenge* (which encapsulates the current block data, a miner's identity tag, and an **epoch seed**) into an output *result matrix*. The PoW condition is that the final result, when compressed to a hash or numeric target, meets the required difficulty threshold (similar in spirit to a hash being below a target in Bitcoin). Unlike a

single hash, however, computing $F$ entails a sequence of heavy matrix operations. We outline the algorithm in steps:

- **Challenge Seed Generation:** Each mining *epoch* (e.g. each block or a group of blocks) provides a 256-bit random seed $S$ derived from known data: for instance, the previous block's hash, the block height (epoch number), and the miner's own identifier (e.g. public key or address). This **identity-bound seed** ensures that each miner's puzzle instance is unique and that a solution can't be reused by others or in future epochs. Formally, one could define $S = \text{Hash}(\text{prev_block_hash} \parallel \text{epoch_ID} \parallel \text{miner_ID})$. The inclusion of miner (or hardware) identity in $S$ ties the work to its origin, adding *replay protection* – a would-be attacker cannot simply reuse a proof found by someone else, since the proof is only valid if the claimed miner's ID matches the one used in $S$.

- **Matrix Initialization:** The seed $S$ is expanded into two initial large matrices $A_0$ and $B_0$ of size $n \times n$ (the dimension $n$ is a protocol parameter defining the matrix size and thus the base work difficulty). For example, $A_0$ and $B_0$ could be filled by pseudorandom number generation seeded by $S$. These starting matrices are effectively random (uniformly distributed) but *deterministic* given $S$, so every honest miner given the same seed will begin with identical $A_0, B_0$. This satisfies the consensus constraint: the "problem instance" is fixed by the blockchain, not chosen by the miner, preventing any miner from picking especially easy matrices. (In other PoUW constructions, miners might choose arbitrary input matrices [8] – here we ensure the challenge is externally imposed and unpredictable.)

- **Iterative Matrix Computation Rounds:** Sky98 then performs $r$ rounds of computation (for some fixed $r$, e.g. on the order of 10–100, chosen to balance security and verification cost). In each round $i = 1,2,\dots,r$, the following operations are applied in sequence:

- **Matrix Multiplication:** Compute the matrix product $C_i = A_{i-1} \times B_{i-1}$. Here $A_{i-1}$ and $B_{i-1}$ are the matrices from the previous iteration (with $A_{0}, B_{0}$ as initialized). The product $C_i$ is an $n \times n$ matrix resulting from multiplying two $n \times n$ integer matrices. This is the most expensive step, requiring $O(n^3)$ arithmetic operations (specifically, $n^3$ multiply-accumulate operations in naive multiplication).
- **Non-linear Transformation ($\sigma$):** Apply an element-wise non-linear function $\sigma(\cdot)$ to $C_i$. That is, for each entry $(u,v)$ in the matrix, compute $D_i(u,v) = \sigma!\big(C_i(u,v)\big)$. The $\sigma$ function is a core design element (detailed in the next section): it introduces *non-linearity* that breaks the simple algebraic relationship of matrix multiplication. In other words, even though $C_i = A_{i-1} B_{i-1}$ is a linear algebraic result, $D_i = \sigma(C_i)$ should be *non-linear* (and non-invertible) enough that one cannot predict or linearize the effect of subsequent rounds. $\sigma$ operates on each matrix element independently (like an activation function in a neural network layer), and is chosen to be deterministic and efficiently computable on hardware (for example, a bitwise operation or a small integer arithmetic function). By design, $\sigma$ ensures that even if one could, say, multiply matrices faster by an algebraic trick, the insertion of this non-linear step every round negates such shortcuts by scrambling the numerical relationships.
- **Masking:** Apply a mask $M_i$ to the matrix $D_i$. The mask $M_i$ can be thought of as either a bitmask applied to each element or a structured matrix that zeros out or permutes certain elements. Masking is another deterministic but hard-to-predict transformation. For instance, a simple mask might zero out all but certain bits of each number (keeping, say, the lower $k$ bits of each $D_i(u,v)$ and discarding the rest), or it might zero out a pattern of entries in the matrix. In Sky98, the mask $M_i$ could be derived from the seed as well (or from an independent sub-seed for each round), so it's *consistent across miners for a given round*, but effectively random and

not controllable by the miner. Let $E_i = D_i \odot M_i$ denote the result of masking (where $ \odot$ is an element-wise operation, e.g. bitwise AND if $M_i$ is a bit mask, or standard matrix multiplication if $M_i$ is a diagonal matrix selecting certain positions). Masking serves to further **break structure and symmetry**, ensuring that intermediate values don't reveal simpler paths to the final result and that any partial information leakage is minimized.

- **Modular Reduction:** Finally, reduce the entries of $E_i$ modulo a fixed integer $q$. Every element of $E_i$ is replaced by its remainder mod $q$: $F_i(u,v) = E_i(u,v) \bmod q$. The modulus $q$ is typically chosen as a power of 2 (e.g. $2^{32}$) or a large prime, to align with hardware-friendly arithmetic. This step accomplishes two things: (a) it bounds the size of matrix entries to prevent uncontrolled growth (after multiplication, entries could be extremely large; reducing mod $q$ keeps values within a fixed range, e.g. 32-bit integers, avoiding overflow issues), and (b) it acts as a simple non-linear transformation as well (modular reduction is nonlinear in the inputs when considered as integers). Notably, using mod $2^{k}$ means that this reduction is essentially just taking the low $k$ bits of each number (if overflow occurs naturally in hardware, this is equivalent to wrap-around mod $2^k$ arithmetic, which many CPUs and GPUs already do for 32-bit/64-bit integers). By performing arithmetic modulo a known $q$, we guarantee **deterministic results** across platforms – no rounding differences – since all arithmetic is effectively integer arithmetic with a fixed word size. This is crucial for consensus: every node verifying the work will get identical $F_i$ matrices bit-for-bit, regardless of whether they use a GPU, CPU, or specialized hardware, as long as they respect the same mod $q$ operations. (This is analogous to Ethereum's EVM using 256-bit integer arithmetic and forbidding floating point to ensure deterministic execution [15].)

- **Prepare Next Round:** The matrices for the next round are then set from $F_i$. For example, we can split $F_i$ into two matrices for the next multiplication: let $A_i = F_i$ and $B_i = \text{Rotate}(F_i)$ or some deterministic function of $F_i$ (such as splitting $F_i$ into two halves if needed, or using $F_i$ and another derived matrix). One simple choice is to use the same matrix for both multiplicands in each round ($A_i = B_i = F_{i}$), effectively repeatedly squaring the state matrix with non-linear perturbations in between; another choice is to alternate using $F_i$ and a fixed initial $B_0$ (so that each round multiplies the evolving matrix by the initial matrix, interspersed with $\sigma$ and mask). The exact update rule can be tuned – the key is that it is deterministic and does not allow trivial repetition or cancellation. For concreteness, one might choose $A_i = F_{i}$ and $B_i = F_{i} \circlearrowleft$ (where $\circlearrowleft$ denotes a fixed cyclic permutation of the matrix elements – effectively a mixing of rows/columns). This means each round's output becomes the input for the next, ensuring a **deep pipeline** of $r$ sequential multiplications separated by non-linear steps.

After $r$ rounds, we obtain a final matrix $F_r$. This matrix (or some hash of it) constitutes the proof-of-work result. Typically, the protocol would require that *H(F_r)* (the cryptographic hash of the final matrix, or perhaps a specific combination of its entries) is numerically below a target $T$ set by the difficulty. This is analogous to Bitcoin: miners in Sky98 will vary their input (in a way we discuss below) until the output meets the difficulty criteria.

- **Nonce and Output Check:** In practice, miners still need a degree of freedom to repeatedly attempt solutions, because the challenge $S$ is fixed per epoch. In Bitcoin, this is achieved with a **nonce** field in the block header that miners increment and hash each time. In Sky98, a similar nonce (or a small vector of nonces) can be incorporated: for example, miners may be allowed to tweak a small portion of the initial matrices $A_0, B_0$ (like some least significant bits or an extra "nonce matrix" added to $A_0$) that does not fundamentally change the problem structure but yields a different final $F_r$. Each distinct nonce gives a different outcome $F_r$, so miners can

try many nonces. Importantly, this nonce space should be large (allowing trillions of attempts if needed, just like a 32-bit Bitcoin nonce) but altering it should not make the problem any easier. Essentially, nonce changes are just slight perturbations in $A_0, B_0$ that lead to different final results, all of which require the full computation to evaluate. The miner iterates over nonce values and runs the $r$ rounds; if the resulting hash $H(F_r)$ is below target $T$, they have found a valid block solution.

This completes the formal definition. We can summarize $F = \text{Sky98Pow}(S, \text{nonce})$ as:

$$
\begin{aligned}
A_0, B_0 &= \mathrm{SeedToMatrices}(S, \mathrm{nonce}) \ , \\
\text{For } i = 1 \text{ to } r &: \\
C_i &= A_{i-1} \cdot B_{i-1} \ , \\
D_i &= \sigma(C_i) \ , \\
E_i &= D_i \odot M_i \ , \\
F_i &= E_i \bmod q \ , \\
A_i, B_i &= \mathrm{DeterministicUpdate}(F_i) \ , \\
\text{Output: } F_r \ ,
\end{aligned}
$$

where $F_r$ is the final $n \times n$ result matrix after $r$ rounds. A block is valid if $\text{Hash}(F_r) < T$ (given current target $T$). All operations are done in integer arithmetic modulo $q$ (e.g. 32-bit unsigned math) to ensure deterministic results across platforms.

## Deterministic Arithmetic and Consensus Safety

A critical requirement for any blockchain PoW (or really any distributed consensus computation) is that the computation is *unambiguous and deterministic*: every honest node must be able to verify the winner's proof and agree on its validity. Sky98 is carefully designed so that despite using complex operations, it remains fully deterministic and yields identical outputs on any machine.

Firstly, we restrict all arithmetic to **integers with fixed width** (or modulo $q$ arithmetic), as noted above. This means there are no floating-point operations, no rounding differences, and no platform-specific math libraries involved. Different CPUs, GPUs or AI accelerators might implement integer multiplication or addition differently under the hood, but modulo $2^{32}$ (for instance) their results are guaranteed to coincide bit-for-bit (assuming no hardware faults). This mirrors the approach used in Ethereum's EVM and many smart contract platforms where *"no floating point, only deterministic opcodes and integer arithmetic"* are allowed [15] to avoid any nondeterminism. In Sky98, even though we draw inspiration from AI workloads (which often use floats), we intentionally constrain the arithmetic to **deterministic integer operations** to satisfy consensus requirements. For example, if a TPU uses bfloat16 or FP16 for speed, it would need to run in an integer-emulation mode for Sky98 or at least ensure the reduced modulo arithmetic matches an exact integer result. (Notably, many AI chips do support integer dot products, e.g. INT8 * INT8 accumulate to INT32, which could be leveraged here.)

Secondly, the algorithm is **fully deterministic given the seed and nonce**. There are no probabilistic choices once the seed is set – the random-looking matrices and masks are all generated by pseudo-random functions from $S$, so every verifier will generate the same $A_0, B_0, M_i$ and so on. The $\sigma$ function and masking operations are also purely functional (no random sampling at verification time). Thus, a verifier can exactly recompute the claimed winning computation (or a subset of it, as we will see) to confirm the result. The consistency of results across heterogeneous hardware is a design goal; we avoid any operation that could be implemented differently by different miners. For instance,

even using something like matrix multiplication itself can be tricky – different BLAS libraries or GPU kernels might produce slightly different floating point round-off results. By sticking to integer math, $A \times B$ is a well-defined operation in $\mathbb{Z}/q\mathbb{Z}$ that is associative and will not vary. We can further specify the multiplication algorithm if needed (e.g. *the canonical matrix multiply algorithm* with $O(n^3)$ operations, not any asymptotically faster algorithm) to remove ambiguity. In fact, one could mandate that miners perform *naïve matrix multiplication* in a fixed iteration order to avoid differences – although practically, as long as the arithmetic is mod $q$, any method to compute $A \times B$ gives the same result, so the main reason to specify algorithm would be to prevent use of faster algebraic algorithms for advantage (a topic we discuss under irreducibility).

In summary, Sky98's computations are as deterministic as a hashing function: given the same input, every honest node will derive the exact same output. This guarantees *consensus safety* – if a miner finds a valid proof, all others can verify it and agree it's valid, with no ambiguity. The use of fixed-point mod $q$ arithmetic and avoidance of any hardware-specific behavior ensures that *"constrained execution environment"* principle [15] of blockchain determinism is upheld.

## The Non-Linear $\sigma$ Function and Algebraic Incompressibility

A distinguishing feature of Sky98 is the introduction of a non-linear transformation $\sigma$ in each round, applied element-wise to the product matrix. The design of $\sigma$ is crucial: it must be simple enough to compute (and to include in hardware), but complex enough to destroy any exploitable algebraic structure in the computation. In other words, $\sigma$ is there to ensure the overall PoW function $F = \text{Sky98Pow}(S,\text{nonce})$ behaves like a *random oracle* from the miner's perspective – *except* by actually executing it, there is no shortcut to predict or partially compute the output.

**Why is $\sigma$ needed?** Consider if we omitted $\sigma$ and masking entirely: then the algorithm would just be $r$ repeated matrix multiplications mod $q$. That would collapse to one big matrix power: $F_r = A_0 \times B_0 \times A_1 \times B_1 \times \cdots \times A_{r-1} \times B_{r-1}$ (depending on how $A_i, B_i$ update – but essentially some long algebraic product of the initial matrices). An enterprising miner could then ask: is there a way to compute this final product faster or more cleverly than doing $r$ separate multiplies? Could one, for instance, multiply all matrices together in a tree (logarithmic depth) or use an optimized matrix exponentiation algorithm? Even more dangerously, could someone find a direct relation between the seed and final matrix that bypasses doing all the work? These possibilities arise because pure matrix multiplication, while computationally heavy, is still an *algebraic* operation – it has structure (linearity in each operand, the associativity that allows rearrangement, etc.). Advanced algorithms (like Strassen's or Coppersmith-Winograd for matrix multiply) achieve speed-ups by exploiting the algebraic structure of linear multiplication. In a worst-case scenario, if $r$ were large and if no non-linearity intervened, a miner might try to use exponentiation by squaring or other techniques to reduce the number of multiplications needed.

The $\sigma$ function is designed to thwart such algebraic shortcuts. By inserting a non-linear, non-invertible function between multiplications, we prevent the miner from collapsing the rounds. Each round's output is not an algebraic polynomial in the previous round's output in any simple way due to $\sigma$. Essentially, $\sigma$ **breaks the chain of linearity**. An analogy can be made to cryptographic ciphers: a pure linear transformation repeated many times can be combined into one linear transformation (and thus is weak), but when you intersperse non-linear substitutions (S-boxes), you get a cipher that is much harder to break. Here, $\sigma$ plays a role akin to an S-box in AES – introducing *confusion* so that the input bits and output bits have a complex relationship.

**Design considerations for $\sigma$:** We want $\sigma(x)$ to be deterministic and efficiently computable on hardware. A simple yet effective choice is to use bitwise operations that are non-linear in arithmetic sense. For example, $\sigma$ might take a 32-bit integer $x$ and do something like: - Rotate $x$ left by a fixed amount, - XOR that with $x$ itself, - Add a constant, - Apply a bitmask.

Concretely, one could define:

$$\sigma(x) = ((x \lll 13) \oplus x) \,\&\, 0xFFFF$$

which means rotate left 13 bits, XOR with original, then keep only the lower 16 bits. This is just an illustrative example – it's nonlinear (XOR is non-linear over $\mathbb{Z}$, and the mask drops higher bits which is like mod $2^{16}$). A more sophisticated $\sigma$ might use a small lookup table (S-box) per byte of $x$ (similar to AES's byte substitution) which is highly non-linear. However, an arbitrary S-box might be costly; bitwise ops are more native to hardware.

The chosen $\sigma$ in Sky98 should also avoid introducing any undesirable structure. For instance, if $\sigma$ were too simple (like $\sigma(x) = x \mod 2^k$, just taking the lower bits), one could argue it's merely a truncation and there might still be some linear relationships in the truncated values. Ideally, $\sigma$ has properties like high algebraic degree, no simple linear or low-degree polynomial representation, etc. This *"breaks algebraic compressibility"*, meaning there is no way to express the effect of $\sigma$ on a large matrix multiplication as a simpler or smaller algebraic problem. Each element after $\sigma$ depends in a complex way on bits of the corresponding product in $C_i$.

It's worth noting that $\sigma$ need not be cryptographically one-way or secure in itself (we rely on the overall structure for security), but it should be *unpredictable* enough. If someone tried to, say, differentiate or linearize the computation, $\sigma$ should derail that effort. In computational complexity terms, $\sigma$ helps enforce that computing $F_r$ is as hard as performing the sequential steps – you cannot compose the $r$ rounds into one closed-form formula of significantly lower complexity. This intuition aligns with the idea of **computational irreducibility** as described by Wolfram: the system *"forces us to experience the event… to achieve an outcome"*, i.e. to get the answer you must simulate the process step by step [11]. By carefully choosing $\sigma$, Sky98 ensures this irreducibility.

In summary, the non-linear $\sigma$ layer is a key innovation that sets Sky98 apart from a naive "just multiply matrices" approach. It injects *complexity* and *chaos* into the computation without adding too much overhead. In our formal description above, $\sigma$ is applied element-wise to $C_i$ yielding $D_i$. Because this is done for every element, it can be massively parallelized (each cell independent), and modern GPUs/TPUs can apply such operations very fast (often at memory bandwidth speeds). Thus, $\sigma$ does not bottleneck the computation but vastly improves security. We will later analyze how $\sigma$ contributes to preventing *shortcut attacks* on the PoW.

## Masking and Modular Reduction

The **masking** step ($E_i = D_i \odot M_i$) and **modular reduction** ($F_i = E_i \bmod q$) collectively serve to add further non-linearity and to control the numeric range of values. We have partially discussed their roles, but we elaborate here on why they are included and how they influence hardware implementation.

**Masking ($M_i$):** Each round $i$, we introduce a mask matrix $M_i$ which is deterministic and derived from the seed (e.g. via a pseudo-random generator that outputs an $n \times n$ mask). The simplest mask is a binary matrix of 0/1 that either keeps or zeroes each corresponding position in $D_i$. This can force some bits or some positions to reset to 0, effectively decorrelating parts of the data from round to

round. More complex masks could permute columns or inject small random constants. The primary reason for masking is to eliminate potential *linearity across elements*. Notice that $\sigma$ itself was applied per element and does not mix data between different matrix entries; thus each matrix entry after $\sigma$ was independent (in terms of computation) from the others. This is fine and by design (for parallelism), but one might worry that a miner could try to solve for each entry independently if they had some linear relation. Masking can add coupling or simply ensure that partial solutions are impossible – for example, if some entries are zeroed out, the miner cannot ignore them as they might carry needed information for the final hash; if some bits of each entry are zeroed, it means the PoW difficulty might focus only on certain bits. Essentially, $M_i$ can be designed to shape the distribution of intermediate values, perhaps to maximize entropy of the next multiplication or to avoid any weak bits.

From a hardware perspective, masking is typically a cheap operation (a bitwise AND with a mask matrix, or setting certain entries to zero). It can be done in parallel across the matrix. Some AI accelerators have limited support for arbitrary masking within the matrix cores, but one can always implement it as element-wise operations outside the main multiply. Since $M_i$ is known in advance for each round, one could also integrate it with $\sigma$ in a single fused non-linear step (for instance, "apply $\sigma$ then zero out these positions").

**Modular Reduction ($\bmod~q$):** We choose a modulus $q$ (such as $2^{32}$). After multiplication, entries in $C_i$ can grow as large as on the order of $n$ times larger than typical entry sizes (since each is a sum of $n$ products). Without reduction, these numbers can become huge after several rounds, which would make verification difficult and also cause inconsistencies if any miner's arithmetic overflows. By reducing mod $q$ at each round, we "reset" the dynamic range of values. This makes every round's outputs lie in a fixed set (e.g. 32-bit integers $0 \ldots 2^{32}-1$). This not only ensures deterministic behavior but also means we can analyze the algorithm in a cyclic group of integers mod $q$.

Using mod $q$ also has a subtle cryptographic effect: it introduces *modular arithmetic*, which is nonlinear (particularly if $q$ is not a power of 2). If $q$ were prime, the multiplication mod $q$ happens in a finite field; however, our $\sigma$ and mask steps break the field structure anyway, so $q$ can be chosen for convenience (power of 2 allows using native 32-bit wraparound, which all hardware naturally does). From the miner's perspective, working mod $2^{32}$ just means they naturally let 32-bit values overflow (which is exactly what typical hardware does in integer ALUs). So there is essentially no extra cost – it's how integer math works. For a verifier, it simplifies things too: they never have to deal with arbitrary precision integers.

In combination, masking and mod reduction make each round's output behave almost like a fresh random matrix (assuming $\sigma$ and the chaotic nature of multiplication did their job). By round $r$, $F_r$ is statistically very close to uniformly random (conditioned on not meeting the target), which is ideal for PoW – it means finding a result below target is purely a brute-force hunt with no structure to exploit other than doing the computations.

## Epoch-Based Timing and Identity-Bound Seeds

Sky98 operates in a blockchain setting, so it must function within the *epoch/block cycle and difficulty adjustment* framework. We clarify how the protocol uses epochs and identity-binding:

- An **epoch** in Sky98 could correspond to a single block interval. Essentially, for each new block to be mined, a new challenge seed $S$ is defined (as described earlier). This means the matrix puzzle changes every block. "Epoch-based timing" refers to the idea that the PoW algorithm might be periodically reparameterized or refreshed to stay aligned with hardware evolution. For

example, every few years (many epochs), the parameter $n$ (matrix size) or $r$ (rounds) might be increased to ensure the work remains sufficiently heavy relative to improved hardware. This is analogous to Bitcoin potentially adjusting parameters if it wanted to remain ASIC-resistant (though Bitcoin doesn't do this, some altcoins have). In Sky98's design, one could imagine an upgrade schedule: e.g., start with $n=128$ matrices, and every 100k blocks increase $n$ by a small amount to push miners towards even more powerful hardware. However, such changes would need consensus agreement. The key point is that within a given epoch (block), the puzzle is fixed.

- **Identity-bound seeds:** We incorporate miner identity in the seed intentionally. This is not typical in Bitcoin PoW (there, anyone can solve and present a hash solution; the identity is only in the coinbase transaction for payout but not tied to the hash itself). By binding the challenge to an identity (like a miner's public key), we enforce that the actual solver of the puzzle is, in a sense, known. This has two main benefits:

- It thwarts *solution theft*: If a miner somehow found a solution and someone else intercepted it, the thief's node could not successfully submit it because the block header would include the thief's own identity which wouldn't match the one used in computing the solution. Essentially, it ties the PoW to the intended miner. In practice, miners could simply include their public key in the block and in $S$, and the verification would check that the solution corresponds to that pubkey. This is somewhat similar to how Proof-of-Stake or certain PoW variants personalize challenges.

- It discourages pooled mining in its naive form: Mining pools traditionally work by having many miners collaborate on the same challenge and whoever finds a solution gives it to the pool operator to publish (the operator being the one whose identity appears in the coinbase). If the PoW is identity-bound, then only the pool operator's identity would be valid in $S$. This means individual workers in the pool can still do the work, but they'd have to use the pool's identity in the computation. That's fine if they trust the pool (standard pools could still operate, the workers essentially computing for the pool's pubkey). However, one could also create schemes where partial proofs are attributable. This topic goes deeper into mining decentralization; suffice to say Sky98 introduces the possibility of more decentralized pool structures since the work itself could be tied to individual identities, possibly preventing one from effortlessly reusing someone else's partial work.

- **Replay protection:** Because each seed $S$ is unique per block (including the previous block hash or height ensures that), no solution can be reused for a future block. Even if two blocks had the same miner identity, the epoch counter or prev-hash would differ, yielding a totally different matrix challenge. Thus, the proof-of-work *instance* is one-time-use. This is standard in PoW, but we highlight it as a design goal that any "useful computation" you incorporate should not accidentally carry over. For example, if miners were allowed to freely choose $A, B$ to multiply, one might fear they precompute many matrix multiplies offline and then try to reuse those results when a similar challenge arises. Sky98 prevents that by deriving $A_0, B_0$ unpredictably from the seed each time.

In summary, the protocol's integration with block production is such that: every block has a publicly known random challenge $S$ (ensuring fairness and unpredictability), optionally personalized by miner identity to secure the provenance of the work, and only a miner expending fresh computation on that challenge can solve it. The difficulty can be adjusted by the network as usual (by raising or lowering the target $T$ which effectively requires more or fewer nonce trials). If miners become too efficient (finding blocks too fast), the target can be made more stringent, and vice versa. Sky98 doesn't change that

mechanism; it simply replaces what the “inner loop” of trying nonces does – now each nonce test is a big matrix computation rather than a quick double-hash.

## Efficient Verification via Sampling and Proof Checking

One potential concern with a heavy PoW like Sky98 is: if it takes, say, $10^{12}$ operations for a miner to find a solution, does every full node on the network also have to do $10^{12}$ operations to verify the block? If yes, that would be impractical – verification should be *orders of magnitude cheaper* than finding the solution (in Bitcoin, verifying a hash solution is trivial – just one hash computation and a comparison).

Sky98 addresses this by providing *verifiable computation* techniques that allow a much faster check than redoing the entire work. We outline two complementary approaches for verification:

### Random Tile Sampling Verification

The first approach is a Monte Carlo spot-check of the matrix multiplication result. Suppose a miner publishes in the block, along with the final hash of $F_r$, also the final matrix $F_r$ itself (or a commitment that allows revealing parts of it). To verify that $F_r$ is correct – i.e., that it indeed equals the defined function of the seed and nonce – a node could randomly sample a small portion of the computation and check it explicitly. We call these portions *tiles*, which could mean a random set of rows, columns, or even individual elements of intermediate matrices.

For example, the verifier can do the following with minimal overhead: - Pick a random index pair $(u,v)$ and a round $i$ to audit (using the block hash or something as randomness to make it unpredictable to the miner in advance). - Ask the miner (or extract from the block, if provided) for the values of the relevant slices of matrices $A_{i-1}$ and $B_{i-1}$ that would produce $F_i(u,v)$. Specifically, to check one element of the product, the verifier needs the entire $u$-th row of $A_{i-1}$ and $v$-th column of $B_{i-1}$ (each of length $n$). The miner could include a *Merkle root* of all intermediate matrices in the block and provide a Merkle proof for those row/column vectors. - The verifier then computes the single dot-product of that row and column (mod $q$), applies $\sigma$ to the result, applies the mask and mod, and checks that it equals the claimed $F_i(u,v)$ (which eventually should match what leads to $F_r$). If this check passes for a handful of randomly chosen elements across different rounds, the verifier gains high confidence that the miner performed the entire computation correctly.

The intuition here is that a fraudulent miner (who tries to fake a proof without doing full work) is very likely to make mistakes in many entries of the matrix product if they didn’t multiply properly. Catching one wrong entry through random sampling would reveal the fraud. If the miner tries to somehow cleverly cheat only part of the computation, the chance that the verifier’s random checks hit that part is significant. By adjusting the number of samples, one can make the probability of undetected fraud astronomically low. For instance, checking, say, 20 random entries could be enough to get error-detection probability over 99.9999% for any significant cheat.

An analogy is checking a large multiplication: if someone claims $X \times Y = Z$ for large matrices, one classic approach is to pick a random vector and check $A(Bv) = Cv$ [16]. In fact, *Freivalds’ algorithm* for verifying matrix multiplication works exactly on this principle – it does a few random linear combinations to catch errors with high probability. In our case, because $\sigma$ and masks complicate direct linear combination checks, verifying intermediate results element-wise via Merkle proofs is a more straightforward approach.

The cost for a verifier doing these checks is very low: computing a dot product of length $n$ is $O(n)$ work, and doing, say, 10 of those is $O(10n)$. If $n = 1024$, that's about 10,240 multiplications – easily done in microseconds to milliseconds on a CPU. The miner's cost was $O(n^3)$ (which is $\sim 1$ billion multiplications for $n=1024$), so verification by sampling is *a tiny fraction* of the original work. This achieves the desired asymmetry: miners do heavy lifting; verifiers do light checking.

The trade-off is that the verifier needs some additional information from the miner – either the entire final matrix (which might be large, but could be on the order of a few MB for $n=1024$, which is not too bad to include in a block or as part of a proof), or at least commitments to intermediate states. One optimization is that the final hash $H(F_r)$ is already a compact commitment to the final matrix. If the verification sampling uses the final matrix's hash as a seed to choose random indices, the miner cannot know in advance which entries will be checked, preventing them from only computing those and skipping others. The miner would have to either include all of $F_r$ or have the ability to quickly recompute any requested entry. Including $F_r$ explicitly might be simplest. While that adds data to each block (e.g. $1024^2$ 32-bit numbers is 4 MB, which is too much), one could use a smaller matrix (like 256x256 = 65k entries, 260kB, somewhat large but maybe manageable). Alternatively, include a commitment and only reveal the sampled entries with proofs.

Another approach is hierarchical: the miner could include the final matrices of each round, hashed in a Merkle tree, so that verification can drill into any round. This is akin to providing a *proof-of-computation* transcript. However, that may be overkill if sampling final output suffices.

## Algebraic Verification via Random Projections

A more elegant verification (inspired by techniques for verifying matrix products) is as follows: To verify that $A_{i-1} \times B_{i-1}$ was done correctly (mod $q$) without checking all entries, the verifier can use a random vector technique: - The verifier picks a random vector $r$ of length $n$ (with entries mod $q$) and multiplies the claimed result matrix $C_i$ by this vector: compute $y = C_i \cdot r$ (which is an $n$-length vector). - Separately, the verifier computes $y' = A_{i-1} \cdot (B_{i-1} \cdot r)$. First compute $z = B_{i-1} \cdot r$ (vector of length $n$), then $y' = A_{i-1} \cdot z$. - If the multiplication was done correctly, we have $y = A_{i-1}(B_{i-1} r) = A_{i-1} \times z = y'$. If the miner cheated in computing $C_i$, then with high probability $y \neq y'$. The verifier only has to do two matrix-vector multiplications, which are $O(n^2)$ each. - We can extend this to include the $\sigma$ and masking by incorporating them into the check: e.g., the miner provides $D_i = \sigma(C_i)$, $E_i = D_i \odot M_i$, and $F_i = E_i \bmod q$. These are hard to verify by linear algebra directly because $\sigma$ and mask are nonlinear. But since those operations are element-wise, the verifier could simply check them on a random subset of entries as described above.

In practice, a combination of both methods might be used: the heavy part (matrix multiplication) can be checked by algebraic means on a random vector [16], and the non-linear parts can be spot-checked. This keeps verification extremely efficient while still catching errors with overwhelming probability.

One can draw parallel to **interactive proofs** or SNARKs for computation: here the miner effectively provides a proof that they did the work (the final hash and any auxiliary info), and the verifier challenges it in a few places. There is ongoing research into *non-interactive proofs of work* where the prover can convince the verifier of a long computation quickly – our approach is a practical instantiation specialized for matrix math.

### Bounded Verification Cost

The outcome of these verification strategies is that the cost for any network node to validate a Sky98 block is on the order of *a few thousand or million operations* at most, which is negligible compared to what the miner did. We ensure that even if $n$ is large (making mining costly), the verification doesn't scale as badly. Typically, if miners are doing $O(n^3)$, the verifier might do $O(n^2)$ or less.

This is critical for decentralization: we do not want only supercomputers to be able to run full nodes. Sky98's design keeps node verification within CPU capabilities by leveraging the structure of matrix operations for spot-checking. This approach is in line with the notion of *proofs of useful work* having to be **verifiable** without redoing the work [17]. In the literature, this sometimes requires sophisticated protocols, but matrix multiplication has nice properties that allow relatively simple verification schemes.

One must also ensure that the size of proofs or data in blocks is not too large. As mentioned, including an entire matrix could be huge, so a more efficient commitment scheme is preferable. For instance, the miner could commit to each round's output via a Merkle root and include only a few Merkle branches for verification. The exact implementation can be tuned for the desired security level vs. bandwidth trade-off.

To summarize, **Sky98 allows fast block verification** by sampling and algebraic checks: - A verifier might verify a block by doing, say, 5–10 dot products of length $n$ (plus associated $\sigma$ calculations for those dot products), which is extremely fast relative to modern processor speeds. - The probability of an invalid proof passing these checks can be made vanishingly small (e.g. $<2^{-60}$) by appropriate sampling. - Thus, although the PoW itself is heavy, it does not bog down the network's transaction validation process significantly.

## Analysis of Security and Computational Irreducibility

In this section, we analyze why the Sky98 PoW is **computationally irreducible** and resistant to shortcut strategies, and how it balances computational and memory demands to deter specialized hardware advantages.

### Computational Irreducibility: No Shortcutting the Work

We informally argue that Sky98's design achieves *computational irreducibility* – that is, the fastest way to obtain the correct output $F_r$ for a given challenge is essentially to perform the $r$ rounds of matrix operations as specified, with no significant shortcut. This property underpins the security of the PoW: if any adversary could compute $F_r$ substantially faster (with less work) than honest miners, they would have an advantage to find blocks more easily, undermining fairness and security.

The irreducibility stems from multiple factors: - **Random-like inputs:** The initial matrices $A_0, B_0$ are essentially random (from the perspective of an attacker, since $S$ is unpredictable until the epoch). There is no structure in these matrices (no special sparsity, no low rank, etc.) that a miner could exploit. They are dense, full-entropy matrices. In fact, this randomness assumption is crucial: known fast matrix multiplication algorithms (like Strassen) can outperform naive $O(n^3)$ on large matrices, but they usually still have to do more operations than $O(n^3)$ when the matrices are filled with arbitrary values – and their constant factors and memory overhead often make them unattractive below very large $n$. By using relatively moderate $n$ (say 256 or 512) and by not allowing any structure the miner could precompute, it's likely that the classical $n^3$ algorithm, which maps well to hardware, is as efficient as any other method. The PoUW literature conjectures that forcing the use of standard matrix multiply

yields optimal security [16], linking it to hardness of solving random linear systems. - **Interleaved Non-linearity ($\sigma$):** As we discussed, $\sigma$ prevents the rounds from being merged or skipped. If a miner thought to do two rounds' worth of multiplication in one go: that would mean trying to compute $A_{i-2} \times B_{i-2} \times A_{i-1} \times B_{i-1}$ directly. But because $\sigma$ was applied to the intermediate $C_{i-1}$, this becomes a fundamentally different computation (it's not just a matrix multiply; it's a complicated function of $A_{i-2}, B_{i-2}, A_{i-1}, B_{i-1}$). There is no known way to evaluate that in fewer operations than doing the first multiply, then applying $\sigma$, then doing the second multiply. In other words, the presence of $\sigma$ *serializes* the process: you *must* finish round $i-1$ to even know what $A_i, B_i$ (or $C_{i}$) are for the next round. - **Masking and mod:** Masking adds a bit of unpredictability across elements, and mod $q$ ensures no over-arching linear relationship spans multiple rounds (because values are taken mod $q$, one can't, for example, set up an easy system of equations over integers covering multiple rounds). - **High algebraic degree:** Each round's computation can be thought of as a function mapping $(A_{i-1}, B_{i-1})$ to $F_i$. Because of $\sigma$ (assuming $\sigma$ is non-linear enough, e.g. involving bit operations), the algebraic degree of this mapping grows. After multiple rounds, $F_r$ is an incredibly complicated polynomial (or bitwise formula) in the bits of $A_0, B_0$. It's implausible that there is a closed-form shortcut to get $F_r$ without basically evaluating that formula.

To put it differently, if a miner attempted an attack such as: - **Precomputation:** They cannot precompute much because the seed $S$ is new each time. At best, if $r$ is small or $n$ is small, they could trade off some storage for speed by precomputing results for some possibilities, but the space of possibilities is astronomically large (even for moderate $n$, $A_0$ has $n^2$ entries each 32-bit, so the state space is huge). - **Parallelization beyond intended:** The miner can parallelize the $n \times n$ multiply internally (which is fine, GPUs do that), and they can attempt multiple nonces in parallel, but they cannot parallelize the *rounds* – those have to happen sequentially. So there is an inherent *depth* of $r$ steps that no parallel machine can circumvent. If $r$ is, say, 10, then there is at least a latency of 10 matrix multiplies that everyone must go through for each attempt. This means even if someone has a very high throughput device, they will still incur a cost in time for those sequential steps. This prevents ridiculously high single-chip hash rates because you can't pipeline an arbitrary number of independent tries without each incurring that sequential cost. (Though in practice miners will still run many attempts concurrently, but each attempt cannot be faster than those 10 rounds.) - **Algorithmic speedups:** Known faster matrix algorithms (Strassen's $O(n^{2.8})$, etc.) rely on recursion and collecting terms in clever ways. They are complicated to implement in hardware and usually involve higher constant overhead. Even if an ASIC miner tried to implement Strassen-like methods, the benefit might be marginal for the sizes we use. And importantly, $\sigma$ and masking between rounds mean you can't just treat the whole 10-round process as one matrix multiplication of size $n \cdot r$ or something – there's no straightforward way to apply a faster algorithm to the entire chain. They would have to apply any fast algorithm round by round at best. If $n$ is small enough (e.g. 256), classical multiplication might actually be optimal; if $n$ is larger, GPUs/TPUs are already highly optimized (with systolic arrays, etc.) to multiply at near hardware peak. So any custom chip trying to outdo a GPU on matrix multiply would basically end up resembling a larger or more efficient systolic array, which is just a better general matrix engine (thus not a wasteful specialization, which is a *good* thing in terms of philosophy).

There is a theoretical assumption here: that computing the Sky98 function is as hard as doing all those operations. This is analogous to the assumption that SHA-256 has no shortcut (except brute force) to find a preimage with certain properties. For SHA-256, that's a well-studied assumption in cryptography. For Sky98's matrix-based function, it's a newer hypothesis but grounded in complexity theory and the hardness of generic linear algebra problems. In fact, one could relate a single round (with no $\sigma$) to solving linear systems; adding $\sigma$ might relate it to something like piecewise-linear or non-linear system solving, which should be even harder.

From the perspective of an adversary trying to *guess* a solution: because Sky98's output appears pseudorandom, the only strategy is trial and error. Each trial requires doing the $r$ rounds. The probability of success per trial is $T/2^{256}$ (if we use a 256-bit hash and target $T$ analogous to Bitcoin's difficulty). Thus expected trials needed is huge, and they can't improve that probability except by doing more trials or speeding up each trial by actual computation.

To reinforce this irreducibility, one could formally argue (though we won't fully do so here) that any significant shortcut would imply solving some underlying hard problem. If someone could compute $F_r$ with, say, half the operations, it likely means they found a way to solve certain equations or invert certain functions faster than believed possible (maybe akin to breaking a cryptographic primitive). By choice of $\sigma$ and masks, we aim to avoid any structure that reduces to a known easy problem.

## Work-Depth Balance and Hardware Efficiency

Sky98's workload is both **compute-intensive** and **memory-involving**, striking a balance that challenges hardware in multiple dimensions: - Each round involves $O(n^3)$ arithmetic operations (if done naively). This is highly parallelizable: there are $n^3$ independent multiply-adds (in theory, one could have $n^3$ processing elements do it in one step, but practically, hardware will do many in parallel over several cycles). GPUs and TPUs excel at exactly this kind of parallel math – for instance, Google's TPU v1 has a 256x256 systolic array that performs 65,536 multiplications (and additions) per cycle [18] [19]. Modern GPUs have thousands of CUDA cores for FMA (fused multiply-add). So the *throughput* of these devices for matrix multiply is enormous. - However, matrix multiply is also **memory-bandwidth bound** if not managed well, because you need to fetch matrix elements from memory. Efficient implementations use blocking and shared memory to reuse data. TPUs use on-chip memory to hold one operand stationary (weights) and stream the other through the systolic array [20] [21]. In Sky98, since the matrices are generated and then thrown away each round (not reused across rounds except the evolving state), miners will have to deal with moving a lot of data. For instance, if $n=1024$, one matrix is 1,048,576 elements; at 4 bytes each, that's 4MB. Multiplying two such matrices might naïvely require reading 8MB and writing 4MB of results. Doing that $r$ times (say 10 times) means 80MB of memory traffic, which is not trivial. A GPU with high bandwidth (e.g. 900 GB/s) can handle it, but it shows that *memory plays a role*. An ASIC that only focused on compute but neglected memory would stall. Thus miners might invest in HBM (High Bandwidth Memory) or large SRAM buffers – again, components that are generally useful for any computing, not just mining. - **Work-depth trade-off:** We mentioned earlier that Sky98 inherently has sequential depth $r$ (number of rounds) that cannot be parallelized away. This is analogous to *Amdahl's Law* considerations: even if 99% of the work is parallel (within each multiply), the 1% that is sequential (the fact you must do round 1 then round 2, etc.) sets a lower bound on how fast you can complete one attempt. If $r$ is small (like 5-10), the latency isn't huge, but it does mean that having, say, 1000 parallel pipelines doesn't let you solve one challenge in 1/1000th the time – it lets you work on 1000 challenges at once, but each still takes the full rounds in latency. In practice, miners care about throughput (solutions per second), not latency, so they will parallelize many attempts. But if $r$ were extremely large, latency could degrade throughput too (due to difficulty adjustments, etc.). Sky98 picks $r$ moderately so that a certain amount of sequential effort is enforced but miners can still pipeline multiple attempts.

- **ASIC vs General-Purpose:** The *ideal* miner for Sky98 would have:
- A large array of multiply-accumulate units (like a matrix engine) to handle the $n \times n$ multiply efficiently.
- Sufficient memory (or on-die storage) to feed those units with matrix data continuously (so they're not idle waiting for memory).
- Fast interconnect or layout to perform the operations like a systolic array or other optimized structure, so as to minimize overhead per multiply.

This description is basically a **Tensor Processing Unit (TPU)** or a high-end GPU. In fact, Google's TPUs were explicitly designed because *"neural network inference is dominated by one operation: $C = A \times B$"* and they built a systolic array to do that with high reuse of data [22] [18]. A Sky98 ASIC miner would almost certainly adopt the same approach: implement a systolic array for matrix multiply, include activation (here $\sigma$) units in each processing element or as a post-process, and perhaps integrate memory for streaming. The difference from a pure AI chip might be slight (maybe different bit-width or the exact $\sigma$ logic). Thus, any money spent to develop such an ASIC is really pushing the envelope of AI chip design – *advancing the state of the art for general matrix computation hardware*. This contrasts with Bitcoin ASICs which consist of tens of thousands of identical SHA-256 hash cores wired together [12] [13] – something that has no use outside cryptocurrency.

Indeed, we can say Sky98's ethos is similar to **ProgPoW** (Programmatic PoW) which aimed to *"utilize almost all parts of commodity GPUs"* so that ASICs gain minimal advantage [23]. In our case, we utilize the matrix units, the memory subsystem, and even require some control logic for sequential rounds. A custom Sky98 ASIC might omit unnecessary features like texture units or graphics pipeline (just as ProgPoW ASIC could omit those [24]), but it could not eliminate the need for lots of general matrix-math capability. The expected ASIC efficiency gain over a GPU might then be relatively small (maybe 2x or so), akin to what ProgPoW designers estimated (~1.1–1.2x) [24], which is a far cry from the 50x–1000x gains specialized SHA256 ASICs have over CPUs/GPUs [12] [13]. This small gap means general-purpose miners (people with GPUs/TPUs) remain competitive, keeping the mining ecosystem more decentralized and accessible.

To illustrate, consider a miner with a top-of-line GPU vs. a hypothetical Sky98 ASIC: - The GPU can perform, say, 200 TFLOPS (10^12 ops) on matrix multiply (like an NVIDIA H100, which is in that ballpark for FP16 or INT8 operations). If our PoW uses 32-bit ops, maybe it's lower, but let's say 20 TFLOPS for 32-bit. It also has ~800 GB/s memory bandwidth. - An ASIC might push this to maybe 40 TFLOPS and 1000 GB/s because it can be more optimized and drop unused features. But it won't be 1000 TFLOPS unless it's extremely large and power-hungry, which has diminishing returns. Meanwhile, that ASIC would essentially double as an AI accelerator with those specs. - Thus, the ASIC might only be a factor of 2 faster per watt or per dollar than a commodity GPU. This is **excellent** compared to Bitcoin, where ASICs are *several orders of magnitude* more efficient, completely driving out GPUs [12].

Furthermore, by including **memory-hard aspects**, we reduce the risk of ASICs that simply throw thousands of cores without enough memory. If $n$ is large, the intermediate data that must be stored and moved is significant. If an ASIC tried to cut corners by using very narrow data paths or minimal memory, it would choke. GPUs by design have large VRAM and wide memory buses. This aligns with insights from memory-hard PoW algorithms (like Ethash) which attempted to use memory accesses to level the field. Sky98 is not purely memory-bound, but it ensures memory is not negligible.

In short, **Sky98 favors architectures with balanced improvements** – better compute *and* better memory and interconnect. It does not reward a trivial brute-force approach (like simply replicating millions of tiny hash cores). Instead, it rewards *innovation in chip architecture* (much like AI demands). This is an intentional contrast with SHA-256 mining, where the only metric is basically number of hashes per second, and the way to scale that is just replicating more circuits and maybe slight optimizations in logic, leading to a *hardware monoculture* and lack of broader applicability [25] [26].

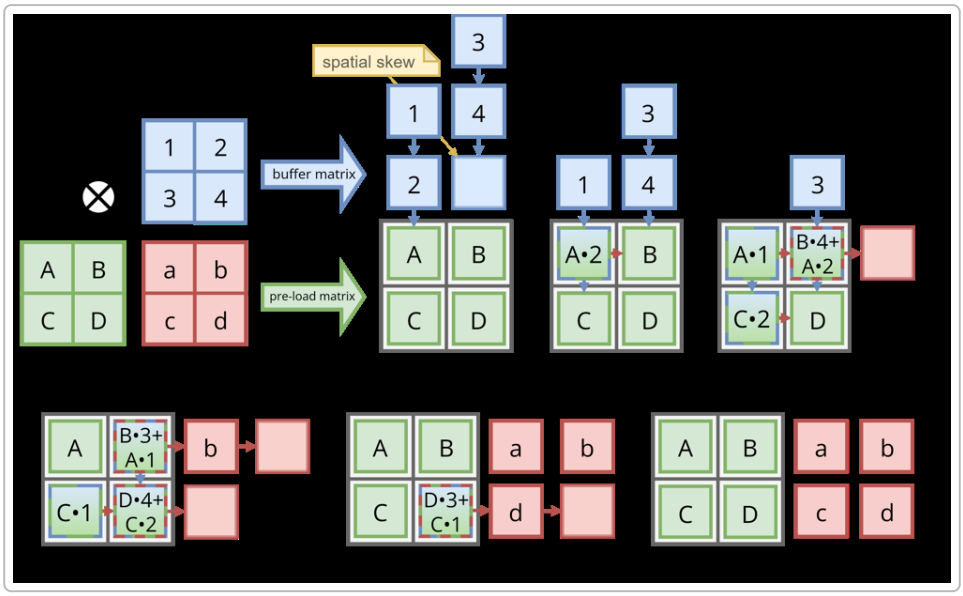


*Figure: A simple 2×2 systolic array illustrating matrix multiplication flow. Systolic arrays are used in AI accelerators like Google's TPU to perform matrix multiply with high efficiency by streaming data through a grid of multiply-accumulate units* [21]*. Sky98's PoW is designed to map onto such hardware structures, ensuring that any mining hardware improvements also advance the state-of-the-art in AI computation.* [18] [27]

Finally, we consider if there are any *attack avenues* unique to matrix-based PoW: - **Low-level math attacks:** Could someone exploit numerical properties, like try to solve the matrix equations instead of doing brute force? Since the matrices are random, solving $A \times B = C$ for random $A, B$ is as hard as multiplying them (no formula helps). And with $\sigma$, the problem isn't even a clean equation – it's more like "find $A, B$ such that after multiplication and weird nonlinear truncations you get some output". This doesn't lend itself to analytic solving. - **Side-channel or reused computation:** Some miners might wonder if partial results can be reused across nonces. Possibly, if two nonces differ only slightly in $A_0$, their computations share some sub-operations. But identifying and exploiting that would be extremely complex and likely not worth it. It's akin to trying to get two different neural network inference to share computation – unless they're identical inputs, you mostly have to do separate work. - **Mining ASIC with AI workload backfeeding:** Interestingly, one *could* attack the spirit of Sky98 by building an ASIC and then *not* using it for AI at all, just for mining. But that's fine – the point is the architecture is still general. Even if they don't actually train neural nets on it, the capability is there. If such an ASIC is developed, it's very likely those same companies could sell it to data centers for AI tasks too, which is a net positive.

## Broader Impact and Conclusion

Sky98 demonstrates a new paradigm for proof-of-work: one that aligns the security of blockchain consensus with the advancement of useful computing. By shifting PoW from arbitrary hashing to *"the kind of math that modern AI models rely on: large matrix operations"* [28], we tap into a synergy between two seemingly disparate fields – blockchain and machine learning hardware. The broader impact is multifold:

- **Energy Efficiency and Utilization:** While Sky98, like any PoW, will consume energy, the computation done is *useful or at least use-inspired*. In an ideal deployment, miners could be running AI workloads (like training neural networks or performing massive matrix computations for scientific projects) and simultaneously producing PoW solutions as a by-product [29]. This *"2-*

*for-1"* use of energy means the effective waste is greatly reduced [6]. Even if miners dedicate hardware solely to Sky98, that hardware remains applicable to other domains. For example, a cluster of Sky98 mining rigs could be repurposed to run machine learning tasks during downtime, or vice versa, creating a more sustainable compute infrastructure ecosystem.

- **Hardware Innovation vs. Stagnation:** Traditional PoW led to what one might call a *computing arms race on a single track* – better and better SHA-256 ASICs, which have limited spillover benefits. This has been described as creating "non-repurposable" hardware [30] [2] and effectively **imprisoning computation** in a useless loop purely to secure the network. As Zawy notes, *"the vast majority of sha256d [ASIC computation] is 'wasted' on bitcoin to satisfy non-repurposeability"* [2]. In contrast, Sky98 encourages an arms race in a multi-purpose direction. Any improvements in Sky98 mining hardware mean improvements in matrix multiplication performance, memory throughput, energy efficiency for arithmetic – all of which are crucial for AI, scientific computing, graphics, etc. This could lead to a virtuous cycle where blockchain mining incentivizes companies to develop more advanced AI accelerators, and those accelerators in turn benefit society at large (outside of blockchain). It breaks the notion that PoW must equal wasted work; instead, PoW can be a driver of *general-purpose* computing progress.

- **Decentralization and Democratization:** By reducing the ASIC advantage and leveraging commodity hardware (GPUs/AI chips that many companies and researchers already have), Sky98 could lead to a more decentralized mining landscape. Large Bitcoin mining is dominated by a few manufacturers and entities that can afford huge ASIC farms. But if mining is done on GPUs and AI clusters, the participant set can be broader (any data center, university lab, or individual with a powerful GPU could contribute). In fact, there's already immense distributed computing power in AI – consider all the GPU farms running training – which could double as a global mining base. The barrier to entry is lower when the hardware has mainstream uses (GPUs) compared to niche SHA256 ASICs which *"there isn't an economic purpose for outside of mining"*, hence only a few invest in them [25]. As ProgPoW's rationale put it, algorithms that favor widely adopted hardware naturally have a wider base of potential miners [25]. This can improve the security of the blockchain by making the mining power more distributed.

- **Security Considerations:** It's worth noting that by aligning with AI compute, one must consider if any external demand for AI compute could interfere with mining security. For instance, if at some time training a lucrative AI model is more profitable than mining, miners might divert hardware to AI tasks, dropping the network hashrate. This is the flip side of repurposability – hardware can leave the mining effort if a more profitable use comes. However, the difficulty adjustment in PoW can handle hashrate fluctuations, and as long as mining remains sufficiently profitable relative to AI tasks (which can be ensured by block rewards/fees balancing out), the security remains. In the long term, one could imagine a symbiotic scenario: miners schedule computations such that when blockchain transactions are heavy and fees high, more hardware goes to mining; when AI demand is higher, some hardware does AI (and maybe that indirectly funds miners who then use profits to mine later). This dynamic is complex but not necessarily negative. It basically ties the security of the blockchain to the overall computing capacity of the world (especially AI computing capacity). Given AI compute is growing exponentially, piggybacking on that trend could mean a blockchain that gets *more secure as AI progresses*, without needing dedicated wastage.

- **Comparison with Traditional Mining:** In SHA-256 mining's heyday, one saw a plateau of innovation: after the initial jump to ASICs, improvements became mostly about scale (more chips, more power usage) rather than fundamental leaps (the SHA-256 function offers no room for algorithmic improvement, only circuit efficiency). That plateau leads to centralization (only

those with latest gen ASIC and cheap power survive). With Sky98, because it's tied to a field (AI hardware) that is still rapidly evolving with huge investments from big tech, there's a continuous push at the cutting-edge. We may see new architectures – photonic matrix multipliers, analog analog neural chips, etc. – which could all be applied to Sky98 mining. The blockchain thus stays at the forefront of technology rather than being secured by essentially frozen-in-time hardware designs. While this introduces *some uncertainty* (e.g., if a radically better matrix computer appears, miners must upgrade), that is not unlike Bitcoin's past where ASIC generations were leaps. The hope is that those leaps benefit more than just mining.

In conclusion, **Sky98** offers a blueprint for a proof-of-work consensus that is *secure, deterministic, and aligned with modern compute needs*. We presented its formal definition and showed that it transforms the mining process into performing deterministic matrix computations analogous to neural network operations. Through non-linear transformations and modular arithmetic, we ensure that the computation is as hard as intended (no shortcuts), embodying computational irreducibility. We demonstrated that the verification of such work can remain efficient, using sampling techniques to avoid burdening validators. We also argued that the algorithm inherently encourages designs that have broad utility (favoring improvements in general-purpose AI accelerators over single-purpose ASICs).

Sky98 is aimed at an audience of hardware engineers, AI chip architects, and blockchain researchers, because its successful implementation requires collaboration across these domains. By adopting a PoW like Sky98, a blockchain network could serve as a continual *benchmark and driver for high-performance computing*. It flips the narrative from "PoW is wasted effort" to **"PoW is pushing useful effort"** – every hash computed is now a matrix multiplied, an operation that the rest of the world also cares about.

As future work, one could explore implementing Sky98 prototypes on existing hardware (GPUs/TPUs) to measure performance and tune parameters ($n$, $r$, $\sigma$ choice, etc.) such that mining is widely accessible yet adequately difficult. Research into the cryptographic hardness (reducing the security to known hard problems) would also bolster confidence – e.g., relating Sky98's hardness to solving random systems of non-linear equations or to the security of an ideal cipher.

Ultimately, Sky98 suggests that the *"proof-of-work versus useful work"* dichotomy can be resolved. The same silicon that powers the latest AI revolution can secure decentralized ledgers, with each domain benefiting from advances in the other [29]. In a world increasingly concerned with the environmental and economic costs of blockchain, Sky98 offers a path forward where that cost is not pure waste but an investment in computing capability that propels other innovations. By adopting matrix-based PoW, blockchains of the future could be secured by the **computing engines of the future** – achieving security through synergy, not waste.

**Sources:**


1. Komargodski, Ilan, and Omri Weinstein. *"Proofs of Useful Work from Arbitrary Matrix Multiplication."* (2023) – Introduces a PoW protocol using matrix multiplication and discusses eliminating energy waste by reusing AI compute [29] [7].

2. Moonlight (AI summary). *"Proofs of Useful Work from Arbitrary Matrix Multiplication – Literature Review."* – Summarizes the cuPOW protocol, noise injection and transcript-based verification for matrix multiplication PoW [8] [16].

3. DelvingBitcoin forum. *"New paper on Proof of Usable Work."* (2025) – Discussion highlighting how AI matrix compute could "merge mine" a blockchain, noting Bitcoin's SHA256 work is largely non-repurposable (wasted) [30] [2].

4. HowAIWorks.ai. *"TPUs vs GPUs vs ASICs: Complete AI Hardware Guide 2025."* – Explains that modern AI relies on large matrix/tensor ops and describes TPU as an ASIC optimized for matrix multiplication with systolic arrays [28] [5].

5. Ethereum Classic ECIP-1070. *"ProgPoW, a Programmatic Proof-of-Work."* – Motivation for ASIC-resistant PoW using commodity GPUs, noting minimal efficiency gains for custom ASIC vs GPU when algorithm uses all GPU parts [23] [14], and describing the huge (~1000×) ASIC advantage on SHA256 due to its simple ops easily wired in ASIC [12] [13].

6. Wolfram, Stephen. *On Computational Irreducibility.* – Concept that certain computations can't be accelerated, requiring step-by-step simulation [11].

7. Cube Exchange. *"Deterministic Execution in Blockchain."* – Emphasizes blockchains avoid nondeterminism: *"No floating point… EVM provides only deterministic opcodes and integer arithmetic"* for consensus across nodes [15].

8. Medium (Owumi Festus). *"Under the Hood of MatMuls: Rust Simulation of TPU's Systolic Array."* – Describes Google TPU's design for efficient matrix multiply (256×256 MAC array) and the motivation to build special hardware for matrix ops as neural nets scaled [18] [27].

9. Wikimedia Commons. *"Weights Stationary Systolic Array Example.png."* – Diagram of a 2×2 systolic array performing matrix multiplication, illustrating how data flows through a grid of PEs (processing elements). This reflects the type of architecture Sky98 mining maps onto [27].

---

[1] [2] [30] New paper on Proof of Usable Work - Protocol Design - Delving Bitcoin
https://delvingbitcoin.org/t/new-paper-on-proof-of-usable-work/1867

[3] [4] [5] [28] TPUs vs GPUs vs ASICs: Complete AI Hardware Guide 2025 | HowAIWorks.ai
https://howaiworks.ai/blog/tpu-gpu-asic-ai-hardware-market-2025

[6] [7] [29] Proofs of Useful Work from Arbitrary Matrix Multiplication
https://arxiv.org/html/2504.09971v4

[8] [9] [10] [16] [17] [Literature Review] Proofs of Useful Work from Arbitrary Matrix Multiplication
https://www.themoonlight.io/en/review/proofs-of-useful-work-from-arbitrary-matrix-multiplication

[11] Formalized math: meet computational irreducibility. | Stephen Wolfram
https://www.linkedin.com/posts/stephenwolfram_formalized-math-meet-computational-irreducibility-activity-7283255742260740096-eDDU

[12] [13] [14] [23] [24] [25] [26] ProgPoW, a Programmatic Proof-of-Work, for Ethereum Classic | Ethereum Classic Improvement Proposals
http://ecips.ethereumclassic.org/ECIPs/ecip-1070

[15] What is Deterministic Execution? Blockchain determinism explained | Cube Exchange
https://www.cube.exchange/what-is/deterministic-execution

18 19 20 21 22 27 Under the Hood of MatMuls: A Rust Simulation of the TPU's Systolic Array | by Owumi Festus | Dec, 2025 | Medium
https://medium.com/@owumifestus/under-the-hood-of-matmuls-a-rust-simulation-of-the-tpus-systolic-array-85de46afec4b